

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Hybrid Movie Recommendation System with User Partitioning and Log Likelihood Content Comparison

YANGYONG MAO¹, KAMPOL WORADIT², (Member, IEEE), KENNETH COSH¹

¹Department of Computer Engineering, Faculty of Engineering, Chiang Mai University, Chiang Mai 50200, Thailand

²Optimized AI Systems for Energy and Environmental Sustainability Research Group, Department of Computer Engineering, Faculty of Engineering, Chiang Mai University, Chiang Mai 50200, Thailand

Corresponding author: Kampol Woradit (e-mail: kampol.w@cmu.ac.th).

Chiang Mai University partially supported this research.

ABSTRACT In the domain of recommendation systems, matrix decomposition is an effective strategy for mitigating issues related to sparsity and low space utilization. The Alternating Least Squares (ALS) method, in particular, stands out for its ability to process data in parallel, thereby enhancing computational efficiency. However, when dealing with an original rating matrix, the ALS method may inadvertently sacrifice some information, leading to increased error rates. To address these challenges, this paper proposes a novel hybrid model that integrates matrix factorization with additional features. Furthermore, it leverages weighted similarity measures and employs advanced log-likelihood text mining techniques. These innovations are designed to tackle cold-start problems and sparsity issues while compensating for information loss to mitigate errors. Under the premise that our model employs consistent evaluation metrics and datasets, comparative analysis against existing models from related literature demonstrates superior performance. Specifically, our model achieves a lower Root Mean Square Error (RMSE) of 0.82 and 0.88, alongside a higher F1 score of 0.94 and 0.92 in two datasets. Our proposed hybrid approach effectively addresses sparsity and mitigates information loss in matrix factorization, as demonstrated by these results.

INDEX TERMS

Recommendation system, log-likelihood, content-based, collaborative filtering, alternating least squares, particle swarm optimization, feature improvement.

I. INTRODUCTION

A Recommendation System (RS) is a tool for helping humans filter an extensive list of items to obtain a small list of items that a user might like. Information overload makes RSs more desirable for people. The popular traditional algorithms in the recommendation field are Content-Based Filtering (CBF) and Collaborative Filtering (CF). Later, many algorithms have been proposed to solve different recommendation problems.

In [1], the sparsity problem is solved by factorizing the sparse rating matrix into two small hidden feature matrices (Matrix Factorization). The predicted value of the element in a sparse matrix is weighted by three methods, namely: 1). Matrix Factorization (MF); 2). Biased Matrix Factorization (BMF); 3). Factor-Wise Matrix Factorization (FWMF). BMF is computed by

the summation of the average score, the bias of the user, the bias of the item, and the predicted value of MF. FWMF is computed by an error between user factors and item factors (see [1]). MF can effectively address the sparsity problem and BMF is primarily used for scalability issues. The FWMF model can be effective in mitigating overfitting problems. However, it's worth noting that similarity overly relies on co-rated items and may overlook the overall rating vector of users and items.

In [2], a hybrid system is built by CBF and CF to obtain recommending results of Top-10 based on Movielens, where the Term Frequency-Inverse Document Frequency (TF-IDF) is used in CF to compute the cosine similarity for obtaining a similar movie list of each movie. A similar user list of each user is obtained in CF by the user's row of the rating matrix. The predicted

value of each element can be computed by the average of similar users/movies. They successfully solved cold start (new movies). However, they do not consider cold-start (new users) and use TF-IDF to generate movie feature vectors for computing movie similarity, where TF-IDF has two problems, namely, 1). Word frequency directly affects the importance of words; 2). Stop words need to be removed individually.

In [3], the authors propose a hybrid system with CBF and CF, whether the clustering of similar movies are found by genre/actor and K-means, and users' historical data can obtain a similar user list of the user. The final recommended result can be computed by the union of clustering and a similar user list. The cold start (new user) problem was mitigated using their location, age, gender, browser, and user device. However, the system does not explicitly address sparsity issues in the recommendation process. Furthermore, the recommended results are limited to movie genre/actors, and there are limitations associated with the use of K-Means clustering. These limitations include the complexity of selecting the optimal K value, potential over-reliance on initial values, and the dependency of clustering results on the size and density of the data.

In [4], a hybrid RS was constructed using CBF and CF on MovieLens, where the similar movie list of each movie is obtained by using cosine based on movies' keywords and properties (CBF). A similar user list of each user was computed by using each row (a user vector) and K-Nearest Neighbors (KNN) after filling in the missing data with zeros in the sparse rating matrix (user-movie). The recommended list was obtained by the union of two similar lists (movies and users). However, it should be noted that the system does not address the cold start problem for new users, and the characterization of movies is limited to keywords and properties, thus not fully resolving the cold start problem for new movies. Additionally, filling in missing data with zeros to alleviate sparsity issues may result in loss of information from the user vector, as discussed in [5].

In [6], the authors propose a similarity algorithm, termed RJMSD (Relevant Jaccard similarity Mixed with Mean Square Distance), for calculating user similarity in recommendation systems. The RJMSD algorithm combines Mean Square Distance (MSD) and Relevant Jaccard similarity (RJaccard) to determine user similarity. Unlike the standard Jaccard similarity, RJMSD takes into account the size of the user data set, thus addressing the cold start problem through weighted consideration of the two similarities. However, it is important to note that MSD is susceptible to sparsity issues, leading to increased similarity errors as the number of common ratings decreases. This limitation highlights the need for further exploration and refinement of similarity algorithms in sparse data environments.

We proposed a novel hybrid recommendation model

based on content and collaborative filtering that combines matrix factorization of alternating least squares for solving sparsity problems. This model maps the rating dimension into hidden feature vectors, avoiding errors caused by the rating dimension. At the same time, it introduces additional feature information to measure similarity and uses the mean of the two dimensions to optimize the matrix decomposition model for reducing information loss during decomposition. In addition, the model also solves the cold start (new users/movies) problem, in which in the content-based algorithm, we no longer use the traditional TF-IDF method, but use a method called hybrid of Log-Likelihood (LL) text mining technology and Continuous Bag of Words (CBOW) model of Word2Vec to alleviate cold starts (new movies), according to the experimental results of this paper, the performance of hybrid of LL and CBOW is better than TF-IDF. Besides, we used user age, gender, and category preferences designed to address cold starts (new users). The overview diagram is shown in Figure 1.

Optimization problems play a key role in improving the accuracy of the proposed hybrid recommendation model. By solving these problems, we can maximize or minimize a specific objective function under given constraints to find the optimal solution. This model mainly utilizes two optimization algorithms: [7]–[9]

1. Grid search (heuristic algorithm): This algorithm uses empirical rules or heuristic strategies to quickly find approximate solutions. Although it cannot guarantee the global optimal solution, it usually provides a sufficiently good solution within a reasonable time, making it particularly suitable for large-scale or complex problems. In this model, grid search is primarily used to obtain the best parameter values.
2. Particle swarm optimization (metaheuristic algorithm): This is an advanced version of heuristic algorithms, with broader applicability and stronger global search capabilities. Metaheuristic algorithms combine the advantages of global and local search strategies. In this model, the particle swarm optimization algorithm is used to determine the optimal weights for age, gender, and category preferences.

The research contributions of this paper are as follows:

- The proposed RS model integrates text features with users' genre preference ratings. This enhances the scalability and accuracy of recommendation systems.
- The introduced Log-Likelihood (LL) weights, which are embedded into content-based submodels, significantly enhance the accuracy of the proposed movie recommendation model.
- Matrix decomposition helps alleviate the negative impact of sparsity while optimizing the model with a mean of two-dimensional to reduce information

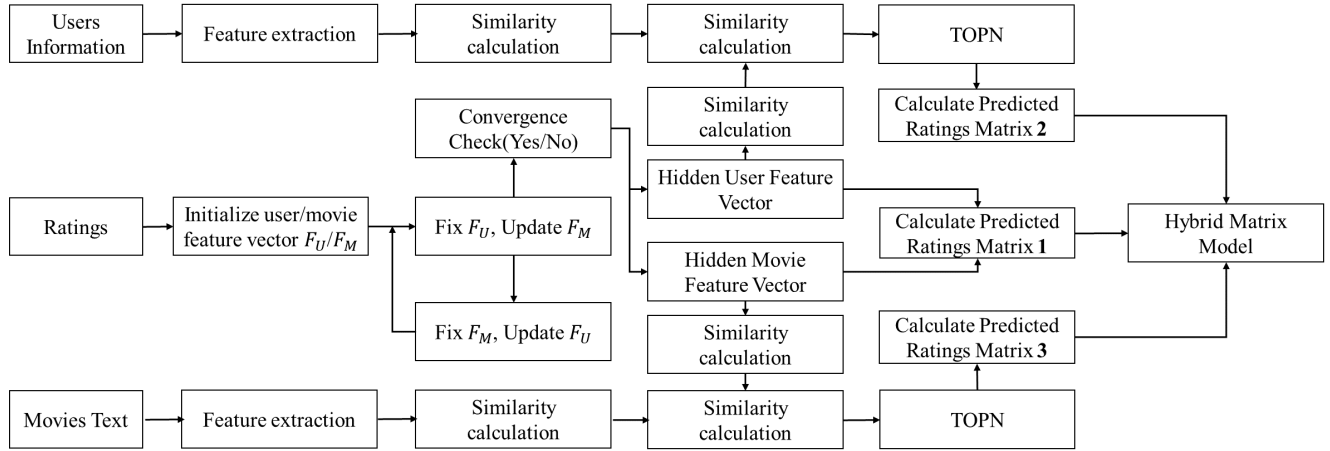


FIGURE 1. Overview Schema of the Proposed Hybrid Recommender System

loss during decomposition.

- The robustness of user/movie similarity measurement is increased by assigning specific weights to multiple similarity metrics and combining them into a single, weighted similarity score.

The organization of the paper is as follows. Section II describes the existing methods. Section III describes the proposed method. Section IV presents the results and discussions, and Section V concludes the paper.

II. EXISTING METHODS

For convenience, this paper briefly describes the existing methods in this section. These methods will be the building blocks of our proposed hybrid method. The symbols and abbreviations used throughout this study are listed in Table 1.

A. CONTENT-BASED FILTERING (CBF)

CBF is one of the most popular conventional algorithms for RS; the filtering relies on the items' properties/keywords and users' profiles based on historical behavioral preferences to construct item feature vectors and user feature vectors. Then, the distance between each user and all movies can be computed by similarity so that the recommended movie list of each user can be obtained [3], [10].

Table 2 shows a feature matrix of users and movies. CBF on recommender systems has obtained user feature vectors of $f_{i,g}^U$ based on historical preferences, and $f_{e,g}^M$ is obtained by extracting from the text of movie e . Then, the similarity can be computed by the distance between $f_{e,g}^M$ and $f_{i,g}^U$.

Figure 2 is an example diagram of CBF in the movie RS. In the figure, "Hellraiser(1987)" and "Hellraiser3 : HellonEarch(1992)" are similar movies. Since user1 likes "Hellraiser3 : HellonEarch(1992)", "Hellraiser(1987)" can be recommended to user1.

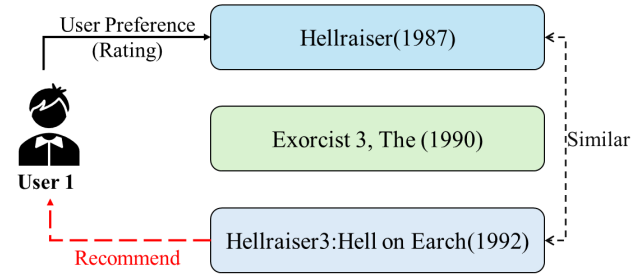


FIGURE 2. Theory of Content-Based Movie Recommendation

The advantage of the system is its ability to mitigate cold start (new movie), and the recommended list of each user does not rely on other users. The Weakness of CBF is that recommended items are based on the current user/item feature vectors. Therefore, the system keeps recommending the same types of items and hardly recommends the items with potential interests [1], [11].

B. COLLABORATIVE FILTERING (CF)

CF has emerged as the predominant and extensively employed algorithm in RS, primarily due to its typically superior recommendation accuracy and broad applicability. CF is generally categorized into two main types: Memory-based CF and Model-based CF.

1) Memory-based CF

Memory-based filtering relies on the item/user feature vectors in the rating matrix to calculate the distance of each user, obtaining a list of similar users for each user and then generating a recommendation list based on the behavioral preferences of those similar users.

TABLE 1. Symbols and abbreviations list.

Symbols/abbreviations	Explanation
N_M	The number of movies
N_U	The number of users
N_R	The number of ratings
N_f	The number of hidden feature
N_g	The number of movie text feature
λ	Regularization coefficients for alternating least squares
$f_{i,g}^M$	A feature element g of movie i
$f_{a,g}^U$	A feature element g of user a
$R_{a,i} \in 0, 1, 2, 3, 4, 5$	The rating is given by user a to movie i
$\hat{R}_{a,i}$	Predicted rating for movie i by user a
PCC	Pearson correlation coefficient
IPCC	Improved Pearson correlation coefficient
JS	Jaccard Similarity
COS	Cosine Similarity
RMSE	Root Mean Square Error
LL	Log likelihood
TF-IDF	Term Frequency-Inverse Document Frequency
R_i	The real rating of i
$\hat{R}_i$	The predicted rating of i
TP	True Positive
FP	False Positive
FN	False Negative
TN	True Negative
CBF	Content-based filtering
CF	Collaborative filtering
UBCF	User-based collaboration filtering
IBCF	Item-based collaborative filtering
maxIter	The maximum number of iterations
a^{age} / b^{age}	The age level of user a/b
a^{gender} / b^{gender}	User a/b 's gender
a^{genre} / b^{genre}	Movie genres liked by user a/b

TABLE 3. Collaborative Filtering Rating Matrix

	Movie 1	Movie 2	...	Movie N_M
User 1	$r_{1,1}$	$r_{1,2}$	...	r_{1,N_M}
User 2	$r_{2,1}$	$r_{2,2}$	...	r_{2,N_M}
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
User N_U	$r_{N_U,1}$	$r_{N_U,2}$	...	r_{N_U,N_M}

TABLE 2. Feature Scores Matrix for Movies and Users in Content-Based Filtering

	Feature1	Feature2	...	Feature N_F
Movie1	$f_{1,1}^M$	$f_{1,2}^M$	...	f_{1,N_F}^M
Movie2	$f_{2,1}^M$	$f_{2,2}^M$	...	f_{2,N_F}^M
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
Movie N_M	$f_{N_M,1}^M$	$f_{N_M,2}^M$	...	f_{N_M,N_F}^M
User1	$f_{1,1}^U$	$f_{1,2}^U$	...	f_{1,N_F}^U
User2	$f_{2,1}^U$	$f_{2,2}^U$	...	f_{2,N_F}^U
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
User N_U	$f_{N_U,1}^U$	$f_{N_U,2}^U$	...	f_{N_U,N_F}^U

Table 3 shows a rating matrix of users and movies. User-Based Collaborative Filtering (UBCF) leverages the shared rating items across rows in the matrix to compute similarity, thereby recommending items that similar users prefer. In contrast, Item-Based Collaborative Filtering (IBCF) relies on the commonly rated items within columns to identify similar items for recommen-

dation [11]–[13].

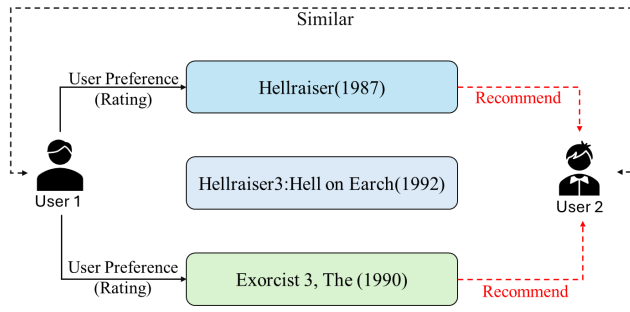


FIGURE 3. Theory of Collaborative Filtering Movie Recommendation

Figure 3 is an example diagram of CF. Based on the figure, it appears that user1 and user2 are similar users. Since user1 likes the movies “Hellraiser(1987)” and “Exorcist3, The(1990)”, it is recommended to suggest these movies to user2, who is also similar in preferences.

However, this model exhibits significant sensitivity to the sparsity of data matrices. In instances where the matrix is excessively sparse, there exists a substantial likelihood that analogous users will not forge connections based on similarity, thereby undermining the efficacy of recommendations.

2) Model-based CF

Model-based CF aims to construct a mathematical model that describes the association between users and items. Among various models, this approach stands out due to its relatively high recommendation accuracy and capability to address issues like sparsity and low space utilization. Notably, matrix factorization is one of the most favored models within this category. Nonetheless, it falls short in addressing the cold start problem and is burdened by high computational complexity.

Among matrix factorization algorithms, Alternating Least Squares (ALS) stands out for its ability to reduce running time costs owing to its parallel computing capabilities [14]. The pseudocode of the ALS algorithm is provided in Table 4. In the iterative process of ALS, acquiring the hidden feature vector of a user/item solely relies on the rating vector of that user/item and remains independent of the rating vectors of other users/items [15].

$$L_{(x,y)} = \sum_{i,j} (r_{i,j} - \hat{r}_{i,j})^2 + \lambda * (||u_i||^2 + ||v_j||^2) \quad (1)$$

$$\hat{r}_{i,j} = \sum_{k \in K} u_{i,k} \cdot v_{j,k}^T \quad (2)$$

In this pseudo-code of ALS, where row 6 and row 7 are simplified by (1) and (2).

TABLE 4. Pseudo-code for Alternating Least Squares Algorithm

ALS algorithm	
Input:	$r_{n,m} \rightarrow UV_{n,m}$ matrix with n rows and m columns $K \rightarrow$ Dimension of each user/movie latent vector $MaxIt \rightarrow$ Number of iterations $\lambda \rightarrow$ Regularization coefficient
Output:	$U_{n,k}$: Characteristic matrix about U ; $V_{m,k}$: Characteristic matrix about V
...	
1	begin
2	Select a non-zero random number and initialize a v matrix
3	$v^0 = np.random.randint(1, 5, (m, k))$ set λ
4	for $t=1$ to $MaxIt$ do
5	for $i=1$ to n do
6	$u_i^t = (v^{t-1} \cdot v_T^{t-1} + \lambda * I)^{-1} \cdot v^{t-1} \cdot r_i$
7	end for
8	$u^t = \sum_{i=1}^n u_i^t$
9	for $j=1$ to m do
10	$v_j^t = (u^t \cdot u_T^t + \lambda * I)^{-1} \cdot u^t \cdot r_j$
11	end for
12	$v^t = \sum_{j=1}^m v_j^t$
	end for
	output : $U_{n,k} = u^{MaxIt}, V_{m,k} = v^{MaxIt}$
	end

TABLE 5. Pseudo-code for Particle Swarm Optimization Algorithm

PSO algorithm	
Input:	$N_P \rightarrow$ Number of particles $D \rightarrow$ Dimension of particles $N_I \rightarrow$ Number of iterations $c1, c2 \rightarrow$ Learning factor $f(x) \rightarrow$ Objective function formula
Output:	Each dimension corresponds to the optimal value of the objective function
...	
1	begin
2	Randomly select positions as velocities for N_P particles, define the initial position of particle i as the local optimal $pBest_i$ of i , and select the optimal particle from N_P particles as the global optimal $gBest$.
3	for $t=1$ to N_I do
4	for $p=1$ to N_P do
5	set $r1, r2, w \in [0, 1]$
6	for $q=1$ to D do
7	$V_{p,q}^t = w * V_{p,q}^{t-1} + c1 * r1 * (pBest_{p,q}^{t-1} - x_{p,q}^{t-1})$ $+ c2 * r2 * (gBest^{t-1} - x_{p,q}^{t-1})$
8	$X_{p,q}^t = X_{p,q}^{t-1} + V_{p,q}^t$
9	end for
10	if $f(x_i)$ better than $pBest_i$ $pBest_i = x_i$
11	end for
12	Update global optimal $gBest$
13	end for
	end

C. PARTICLE SWARM OPTIMIZATION (PSO) ALGORITHM

PSO is an evolutionary computation algorithm we use to obtain the weight of age, gender, and movie genre preference. It works by particle information sharing and lets the movement of particles searching for the optimal solution in the group evolve from disorder to order for obtaining the optimal solution [16]. The pseudo-code of the PSO algorithm is illustrated in Table 5.

D. LOG-LIKELIHOOD (LL)

TABLE 6. Contingency table for word frequencies

	CORPUS 1	CORPUS 2	TOTAL
Freq of word	a	b	a+b
Freq of other word	c-a	d-b	c+d-a-b
TOTAL	c	d	c+d

This study explores the innovative application of the LL method for text mining in hybrid recommendation systems. Unlike conventional techniques such as Term Frequency-Inverse Document Frequency (TF-IDF), LL employs a statistical approach to analyze term co-occurrence, effectively identifying significant patterns in textual data. This method enhances feature extraction precision while addressing challenges related to sparsity and contextual relevance. Terms that appear too frequently tend to have lower LL values. The calculation is defined by equations (3) and (4). The calculation formula is (3) and (4). The formula of LL is given by

$$E_i = \frac{N_i * \sum_i O_i}{\sum_i N_i} \quad (3)$$

$$LL = 2 * \sum_i O_i * \ln \frac{O_i}{E_i} \quad (4)$$

In our case $N_1 = c$, and $N_2 = d$. So, for this word, $E_1 = c * (a + b) / (c + d)$ and $E_2 = d * (a + b) / (c + d)$. $LL = 2 * ((a * \log(a/E_1)) + (b * \log(b/E_2)))$ [17], [18]. In our experiments, we will apply and evaluate TF-IDF and LL to demonstrate the superiority of LL.

E. CONTINUOUS BAG OF WORDS (CBOW)

When using LL to calculate movie similarity through similarity measures, the similarity between two movies may be significantly affected by the sparsity of the movie vectors, thereby degrading prediction performance. To address this problem, we plan to introduce the CBOW algorithm into the movie similarity calculation. CBOW is a model of Word2Vec that predicts target words based on context words. Its working principle involves inputting the context from a piece of text (specifically, the words within a certain range before and after the target word), converting these context words into vectors, and averaging them. Then, a simple neural network is used to map the average vector to the probability

distribution of the target word. Ultimately, the goal of CBOW is to maximize the probability of correctly predicting the target word given the context, thereby effectively training a word vector representation [19].

F. SIMILARITY CALCULATION

Similarity algorithms need to be used between users/items. Commonly used similarity calculations in the recommendation field include cosine similarity calculation, Pierce similarity calculation, RV similarity calculation, etc.

TABLE 7. The purpose and reference of similarity measurement

Similarity measure	The propose of formula	Reference
COS(5)	Hidden feature similarity*	[20]
RV(6)	Movie text similarity	[17]
IPCC(8)	Hidden feature similarity*	[21]
ED(9)	Hidden feature similarity*	[22]
JS(10)	User genre preference	[23]
* Feature similarity selection		

1) Cosine similarity

Cosine similarity (COS) quantifies the degree of similarity by computing the cosine of the angle between two vectors. The COS formula between two vectors, a and b, is given by

$$\text{COS}_{a,b} = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|} \quad (5)$$

where the resulting score falls between -1 and 1. A score closer to 1 indicates a higher degree of similarity between the two vectors [20].

2) RV coefficient

The RV coefficient is used to measure the similarity between vectors with different dimensions. The RV coefficient formula between vector a, and vector b is as follows.

$$\text{RV}_{a,b} = \frac{\sum_{g=1}^s (a_g \cdot b_g)}{\sqrt{\sum_{g=1}^n (a_g)^2} * \sqrt{\sum_{g=1}^m (b_g)^2}} \quad (6)$$

The result of RV calculation is between 0 and 1, and the closer the result is to 1, the higher the similarity between the two vectors [1].

3) Improved Pearson Correlation Coefficient

Pearson Correlation Coefficient calculation remains robust against outliers and extreme values, ensuring stability in similarity calculations.

$$\text{PCC}_{a,b} = \frac{\sum_{i \in I_a \cap I_b} (R_{a,i} - \bar{R}_a)(R_{b,i} - \bar{R}_b)}{\sqrt{\sum_{i \in I_a} (r_{a,i} - \bar{R}_a)^2} * \sqrt{\sum_{i \in I_b} (R_{b,i} - \bar{R}_b)^2}} \quad (7)$$

$$\text{IPCC}_{a,b} = \frac{\sum_{i \in I_a \cap I_b} [(R_{a,i} * \bar{R}_a) - (R_{a,i} * \bar{R}_i)] * [(R_{b,i} * \bar{R}_b) - (R_{b,i} * \bar{R}_i)]}{\sqrt{\sum_{i \in I_a} [(R_{a,i} * \bar{R}_a) - (R_{a,i} * \bar{R}_i)]^2} * \sqrt{\sum_{i \in I_b} [(R_{b,i} * \bar{R}_b) - (R_{b,i} * \bar{R}_i)]^2}} \quad (8)$$

Nevertheless, if numerous outliers and extreme values are present in the PCC algorithm, it may lead to significant deviations in the results. In [21], the author proposes an improved PCC algorithm that considers deviations in two dimensions, thereby enhancing calculation accuracy. The calculation formula is (8).

4) Euclidean Distance

Euclidean Distance (ED) is utilized for computing the distance between two vectors and serves as an indicator of the similarity between the two vectors. The similarity between a and b can be measured as in (9).

$$\text{ED}_{a,b} = \frac{1}{1 + \sqrt{\sum_{g=1}^k (f_{a,g} - f_{b,g})^2}} \quad (9)$$

The ED typically ranges greater than 0. However, for comparison purposes in similarity measures, the Euclidean distance similarity is transformed to a range between 0 and 1.

5) Jaccard similarity

Jaccard Similarity (JS) quantifies the proportion of elements shared between the sets, by dividing the size of their intersection by the size of their union [24]. The formula is given by

$$J_{a,b} = \frac{|a \cap b|}{|a \cup b|} \quad (10)$$

G. EVALUATION INDICATORS

Different evaluation metrics will be employed during the model training in this study, comprising precision, recall, F1 score, and RMSE. Precision and recall assess the specificity and sensitivity of an RS by measuring the frequency of correctly recommended movies and are mathematically expressed as follows:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (11)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (12)$$

However, there exists a trade-off between precision and recall, such that increasing one typically results in a decrease in the other. To address this problem, RSs also employ the F-measure as one of the performance evaluation metrics. The formula is given by

$$F1 = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \quad (13)$$

Each of these three evaluation metrics emphasizes different aspects of model performance, and their combined

use can provide a more comprehensive evaluation of the model's performance.

Additionally, the Root Mean Square Error (RMSE) is used to evaluate prediction accuracy by averaging the squared errors, giving more weight to larger errors and less weight to smaller errors [25]. The mathematical representation of the RMSE is as follows:

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^N (R_i - \hat{R}_i)^2}{N}} \quad (14)$$

Where N denotes the number of ratings [26].

III. PROPOSED HYBRID METHOD

Figure 4 summarizes the research framework of the paper. Our system contains four components:

Step 1) Matrix factorization of ALS (see Table 4) that transforms a sparse matrix into two low-dimensional submatrices, namely, hidden user-feature matrix and hidden movie-feature matrix. The predicted rating matrix 1 can be calculated by multiplying them. Besides, two hidden feature matrices can be used to obtain predicted matrices of step 2 and step 3.

Step 2) The user's age, gender, and preferences for movie genres are leveraged to alleviate the user cold start issue. The optimal weighting is determined using PSO to capture demographic similarities among users. The weighted hidden user feature vector is then utilized to generate the prediction rating matrix 2 via TOP-N.

Step 3) The movie's text feature (LL) is utilized to address the movie cold start problem and enhance recommendation diversity. By leveraging its similarity matrix and weighting the implicit movie feature similarity matrix, we can generate a prediction rating matrix 3 using the TOP-N.

Step 4) The weighting of the three prediction rating matrices is used to alleviate the problem of information loss during matrix factorization. The weighted results will be compared with other recommendation models using Precision, Recall, F1, and RMSE evaluation indicators to demonstrate the model's superiority.

A. DATASET

The paper utilized the Movielens 100K (ML-100K) and Movielens 1M (ML-1M) databases for model training, which includes user ID, user age, user gender (M male, F female), user occupation, movie title, movie ID, movie genre and user rating (1-5 rating) [11]. The textual information about the movies in Movielens can be obtained from the Wikipedia movie database. However, we needed to examine each movie one by one to determine

TABLE 8. Details for each dataset

Dataset	N_U	N_M	N_R	Link
ML-100K	943	1,682	100,000	https://grouplens.org/datasets/movielens/100k/
ML-1M	6,040	3,900	1,000,209	https://grouplens.org/datasets/movielens/1m/
WIKIX	-	300,000+	-	https://library.kiwix.org/viewer#wikipedia_en_movies_maxi_2024-03

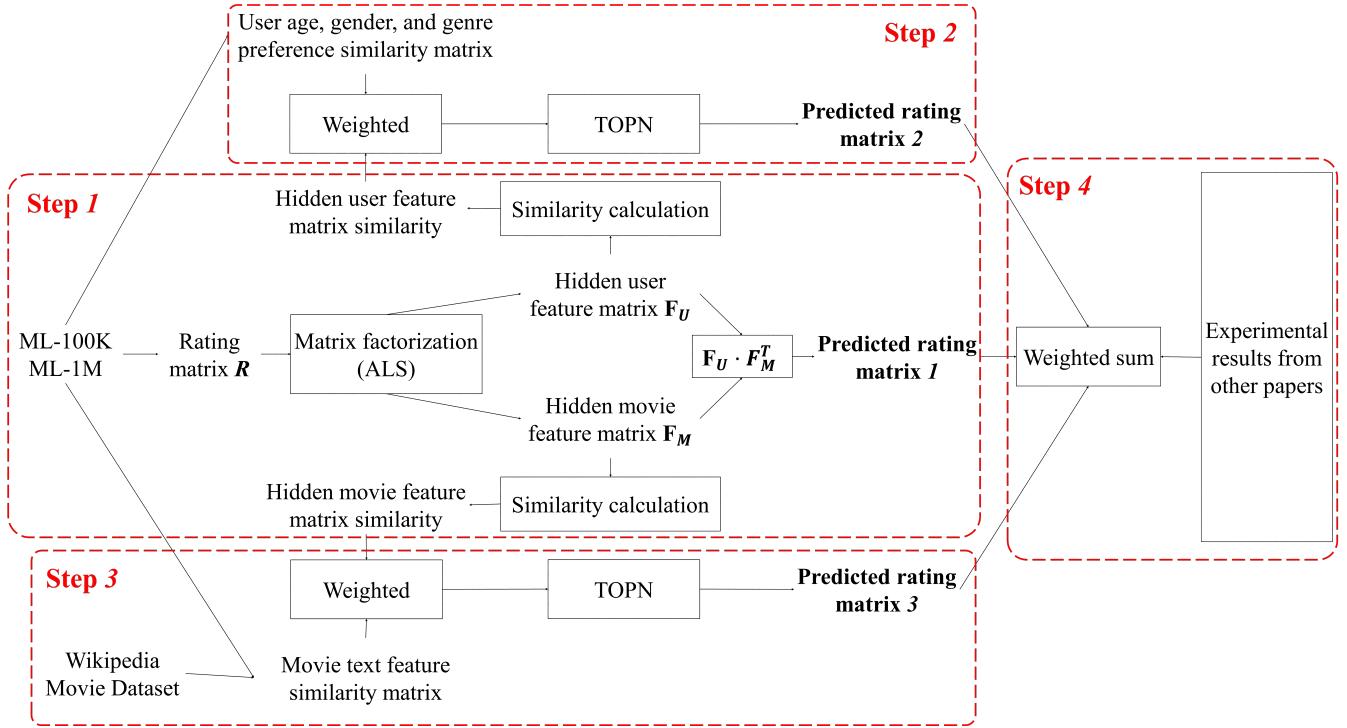


FIGURE 4. Proposed Hybrid Model Framework

the accuracy of the two database connections. Refer to Table 8 for specific details and hyperlinks related to datasets.

In the modeling process of machine learning, the data set is generally divided into a training set (80%) and a test set (20%). The training set is used to modify parameters to generate the model, and the test set is used to evaluate the model. To prevent the problem of over-fitting¹, this paper uses K-Fold cross-validation, which randomly divides the data into K equal parts, each subset is used as a verification set, and the remaining K-1 subset data are used as a training set, K evaluation results can be obtained through K verification sets, and the average of the results is found to improve the accuracy of the evaluation results.

B. MATRIX FACTORIZATION OF ALS

As shown in step 1 in Figure 4. After setting hyperparameters N_f , λ , and $\maxIter$, the original sparse

rating matrix is decomposed into two low-dimensional submatrices. The predicted rating matrix 1 is given by

$$\hat{R}_{a,i} = F_u \cdot F_m^T \quad (15)$$

Three similarity measures (ranging from 0 to 1) are used to find the best similarity calculation method. The formulas for predicting similar users and movies are following as (21) and (24). In Table 5, the evaluation results compare TOP - N ($N \in 1, 2, 3, \dots, 20$), where the COS algorithm performed the best.

C. GETTING PREDICTED RATING MATRIX 2 FROM SIMILAR USERS

As shown in step 2 in Figure 4, we use the user's age, gender, and movie genre preferences as additional feature points for users. New users without ratings can still calculate the similarity for new users according to additional feature points.

In the Movielens movie database, the ages of the users are grouped into seven groups. Then, the similarity of age between users, based on the age levels defined in Table 9, can be calculated using (16). We calculate

¹The model can match the training data well, but cannot predict data outside the training set well

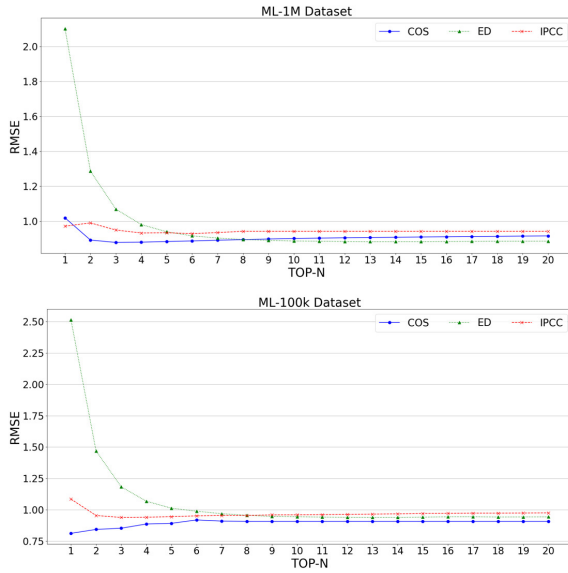


FIGURE 5. Comparison of similarity measures for implicit feature vectors

TABLE 9. User Age Levels in the Movielens Database

Age in Movielens	Age Level
1-18	1
18-25	2
25-35	3
35-45	4
45-50	5
50-56	6
56+	7

the similarity between users with gender using (17). To alleviate the difficulty in obtaining demographic data, this paper introduces users' preferences for movie genres. Movie genre preferences of users' likes are not directly shown in the Movielens database. Therefore, the genres need to be obtained using movies with ratings higher than 3 (only ratings of 4 or 5 points on the Likert scale indicate liking) in the trainset. These obtained genres are considered as users' movie genre preferences in the trainset. Subsequently, the similarity of movie genre preferences between users is calculated using the formula (18).

$$\text{Sim}_{a,b}^{\text{age}} = \begin{cases} \frac{1}{|a^{\text{age}} - b^{\text{age}}|}, & |a^{\text{age}} - b^{\text{age}}| > 1 \\ 1, & |a^{\text{age}} - b^{\text{age}}| \leq 1 \end{cases} \quad (16)$$

$$\text{Sim}_{a,b}^{\text{gender}} = \begin{cases} 0, & a^{\text{gender}} \neq b^{\text{gender}} \\ 1, & a^{\text{gender}} = b^{\text{gender}} \end{cases} \quad (17)$$

$$\text{Sim}_{a,b}^{\text{genre}} = \frac{a^{\text{genre}} \cap b^{\text{genre}}}{a^{\text{genre}} \cup b^{\text{genre}}} \quad (18)$$

After obtaining the three similarities above (age, gender, movie genre preferences), we combine them with the weighting algorithm via (19).

$$\text{Sim}_{(a,b)} \text{UI} = L_1 \text{Sim}_{a,b}^{\text{age}} + L_2 \text{Sim}_{a,b}^{\text{gender}} + L_3 \text{Sim}_{a,b}^{\text{genre}} \quad (19)$$

where L_1 , L_2 , and L_3 denote the weight of age, gender, and movie genre preferences, for which $L_1 + L_2 + L_3 = 1$. The fitness function is the RMSE function (14). The optimal number of initialized particles is 100, and the best number of iterations is 50. The optimal solutions for L_1 , L_2 , and L_3 are 0.31, 0.27, and 0.42, respectively.

The Cosine similarity measure of the hidden feature vector in Step1 and the weighted formula of $\text{Sim}_{a,b} \text{UI}$ is given by:

$$\text{Sim}_{a,b} = \alpha_u \text{Sim}_{(a,b)} \text{HFM}_u + \beta_u \text{Sim}_{(a,b)} \text{UI} \quad (20)$$

Where $\alpha_u + \beta_u = 1$, This paper assigns weight values to α_u from $\{0.1, 0.2, \dots, 0.9\}$ to determine the optimal value and calculates out the predicted rating matrix 2 using TOP-N and the formula (21).

$$\hat{R}_{a,i} = \bar{R}_a + \frac{\sum_{b \in \text{Top-n}} \text{Sim}_{a,b} (R_b - \bar{R}_b)}{\sum_{b \in \text{Top-n}} \text{Sim}_{a,b}} \quad (21)$$

D. GETTING PREDICTED RATING MATRIX 3 FROM SIMILAR MOVIES

As shown in step 3 in Figure 4. Similar movie text can effectively alleviate the movie cold start problem while compensating for the hidden movie feature matrix caused by information loss.

Figure 6 shows a movie article "ToyStory(1995)" in the Wikipedia movie database. The text features of the movie will be obtained via data mining.

The movie feature vector with LLs of Figure 6 is extracted by (4) and is shown in Table 10, where it is sorted by the frequency of words in the movie. The words that appear often in the movie include **the**, **and**, and **toy**. Obviously, **the**, **and** are stop words that are not important, and the LL of both **the** and **and**, are **0.17** and **0.72**. Their effect is negligible compared to the largest LL of **2078**.

The feature extraction using Log Likelihood (LL) can result in varying feature dimensions for different movies, which may significantly affect the RV coefficient similarity calculation due to sparsity issues. To address this, we employ the Continuous Bag of Words (CBOW) model in Word2Vec, combined with LL feature vectors, to update the movie's feature vector using (22). The RV coefficient (6) is then used to calculate movie similarity.

Toy Story

Toy Story is a 1995 American computer-animated comedy film directed by John Lasseter (in his feature directorial debut), produced by Pixar Animation Studios and released by Walt Disney Pictures. The first installment in the *Toy Story* franchise, it was the first entirely computer-animated feature film, as well as the first feature film from Pixar. It was written by Joss Whedon, Andrew Stanton, Joel Cohen, and Alec Sokolow from a story by Lasseter, Stanton, Pete Docter, and Joe Ranft. The film features music by Randy Newman, was produced by Bonnie Arnold and Ralph Guggenheim, and was executive-produced by Steve Jobs and Edwin Catmull. The film features the voices of Tom Hanks, Tim Allen, Don Rickles, Jim Varney, Wallace Shawn, John Ratzenberger, Annie Potts, R. Lee Ermy, John Morris, Laurie Metcalf, and Erik von Detten.

This article is about the 1995 film. For the franchise, see *Toy Story (franchise)*. For other uses, see *Toy Story (disambiguation)*.

Taking place in a world where toys come to life when humans are not present, the plot focuses on the relationship between an old-fashioned pull-string cowboy doll named **Woody** and a modern space cadet action figure, **Buzz Lightyear**, as Woody develops jealousy towards Buzz when he becomes their owner Andy's favorite toy.

Following the success of *Tin Toy*, a short film released in 1988, Pixar was approached by Disney to produce a computer-animated feature film told from a small toy's perspective. Lasseter, Stanton, and Docter wrote early story treatments, which were rejected by Disney, who wanted *Toy Story*'s tone to be "edgier". After several disastrous story reels, production was halted and the script was rewritten to better reflect the tone and theme Pixar desired: "toys deeply want children to play with them, and ... this desire drives their hopes, fears, and actions". The studio, then consisting of a relatively small number of employees, produced the film under only minor financial constraints.

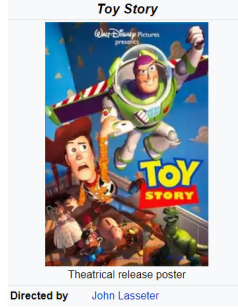


FIGURE 6. The Text of the Movie *Toy Story* (1995) in the Wikipedia Movie Database

TABLE 10. Word Frequency Statistics and Log-Likelihood Values for Movie *Toy Story* (1995)

ROW ID	WORDS	FREQ	LOG LIKELIHOOD
1	the	893	0.176878
2	and	313	0.727359
3	toy	287	2078.108
4	stori	230	727.6727
5	film	213	1.647865
6	from	141	21.62963
7	anim	124	377.8487
8	wa	122	3.495931
9	origin	122	63.49173
10	retriev	108	12.64055
11	for	107	0.193453
12	archiv	107	70.45728
13	with	106	3.952148
14	pixar	79	611.1305
15	releas	73	29.25629
⋮	⋮	⋮	⋮

$$V_{\text{movie}} = \frac{\sum_{g=1}^n LL_{(a_g)} V_{\text{word2Vec}(a_g)}}{\sum_{g=1}^n LL_{(a_g)}} \quad (22)$$

Where $V_{\text{word2Vec}(a_g)}$ denotes word embeddings of a_g ; $LL_{(a_g)}$ represents Log-likelihood values of a_g (4).

Figure 7 presents the RMSE of the prediction results for the hybrid models combining Word2Vec with LL and Word2Vec with TF-IDF in the content-based sub-model. The figure highlights the effectiveness of Word2Vec in mitigating the impact of sparsity on LL and TF-IDF similarity calculations. The combination of Word2Vec and LL consistently outperforms LL, TF-IDF, and the

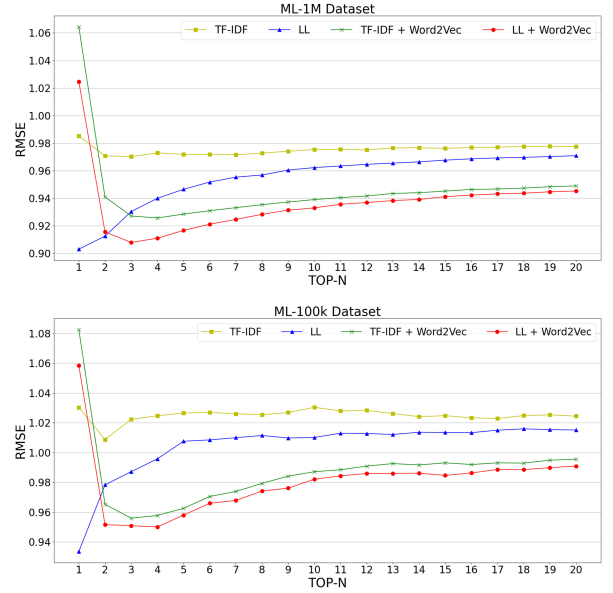


FIGURE 7. Evaluating RMSE of Text Representation Methods: TF-IDF, LL, TF-IDF+Word2Vec, and LL+Word2Vec

combination of TF-IDF with Word2Vec. Additionally, LL proves to be superior to TF-IDF in the model.

Its weighted formula with the implicit movie feature vector is given by the formula (23).

$$\text{Sim}_{i,j} = \alpha_m \text{Sim}_{(i,j)} \text{HFM}_m + \beta_m \text{RV}_{(i,j)} \quad (23)$$

Where $\text{Sim}_{(i,j)} \text{HFM}_m$ denotes the COS similarity (5) between movies i and j in the hidden movie feature vector matrix (from step 1).

The predicted rating matrix 3 can be obtained by TOP-N and formula (24).

$$\hat{R}_{a,i} = \bar{R}_i + \frac{\sum_{j \in \text{Top-n}} \text{Sim}_{i,j} (R_j - \bar{R}_j)}{\sum_{j \in \text{Top-n}} \text{Sim}_{i,j}} \quad (24)$$

E. WEIGHTING OF THREE PREDICTIVE RATING MATRICES

Three predicted rating matrices will be obtained by step 1, step 2, and step 3. To assess the compensatory effect of prediction scoring matrices 2 and 3 on information loss during matrix decomposition, various weighting values will be employed for evaluation. The weighting formula is given by

$$\hat{R}_{(a,i)} = \alpha \hat{R}_{(a,i)}^{s1} + \beta \frac{\hat{R}_{(a,i)}^{s2} + \hat{R}_{(a,i)}^{s3}}{2} \quad (25)$$

Where $\alpha + \beta = 1$, and $\hat{R}_{(a,i)}^{s1}$, $\hat{R}_{(a,i)}^{s2}$, $\hat{R}_{(a,i)}^{s3}$ respectively denotes Predicted rating for movie i by user a in step 1, 2, 3. Besides, the value range of the weight value α will be defined in $\{0.1, \dots, 0.9, 1\}$.

F. COMPLEXITY ANALYSIS

In step 1, the complexity of ALS in row 6 of Table 4 is $O(N_U(N_M N_f^2 + N_f^3))$, and the complexity in row 9

is $O(N_M(N_U N_f^2 + N_f^3))$. In a complete iteration, all users and items need to be updated. Thus, the total complexity of one iteration is:

$$O(N_U(N_M N_f^2 + N_f^3)) + O(N_M(N_U N_f^2 + N_f^3)) \\ = O(N_U N_M N_f^2 + (N_U + N_M) N_f^3) \quad (26)$$

In step 2, the complexity of calculating the similarity of user age, gender, and movie type preferences is $O(N_U^2)$. After applying the PSO algorithm, the complexity is $O(N_t * N_P N_D)$. The complexity of obtaining TOP-N recommendations is $O(N_M \log N_M)$. Therefore, the total complexity of step 2 is:

$$O(N_U^2) + O(N_t * N_P N_D) + O(N_M \log N_M) \quad (27)$$

In step 3, the complexity of generating movie features using LL is $O(N_M N_g)$, the complexity of calculating movie similarity is $O(N_M N_g)$, and the complexity of obtaining TOP-N recommendations is $O(N_M \log N_M)$. The total complexity of step 3 is:

$$O(N_M N_g) + O(N_M N_g) + O(N_M \log N_M) \\ = O(2N_M N_g + N_M \log N_M) \quad (28)$$

In summary, the total complexity of steps 1, 2, and 3 is approximate:

$$O(N_U N_M N_f^2 + (N_U + N_M) N_f^3) \\ + O(N_U^2) + O(N_t * N_P N_D) \\ + O(N_M \log N_M) + O(N_M d + N_M \log N_M) \quad (29)$$

Since $O(N_U N_M N_f^2 + (N_U + N_M) N_f^3)$ is the dominant term, the overall complexity mainly depends on the number of users, movies, and features. Therefore, the total complexity is:

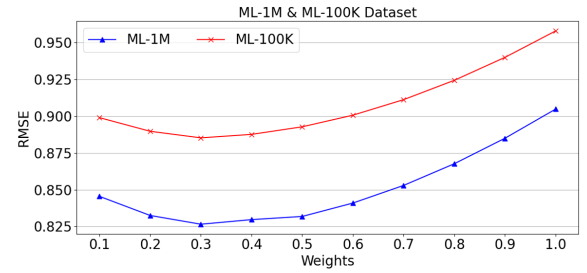
$$O(N_U N_M N_f^2 + (N_U + N_M) N_f^3) \quad (30)$$

The feature dimension (N_f) in the model remains fixed and does not automatically adjust with the increase in the number of users or movies. Instead, it is a pre-defined hyperparameter determined by the model developer during the design phase. The computational complexity of the model increases linearly with the number of users (N_U) and movies (N_M), which enables the model to scale effectively to larger datasets. Furthermore, in most real-world application scenarios, models with linear complexity are capable of processing data for new users or movies more efficiently, while maintaining high prediction accuracy.

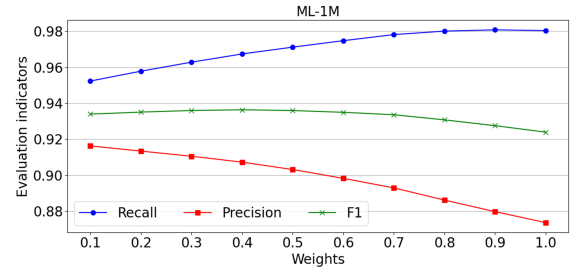
IV. RESULTS AND DISCUSSIONS

This section conducts a comprehensive analysis of various weight values in ML-1M/100K to determine the optimal weight value for user/movie interactions. Furthermore, the study compares our model with existing models from the literature, showcasing the superior performance of our approach.

The similarity weight diagram illustrating the relationship between user demographics with genre preferences in Step 2 and the hidden user feature vector in Step 1 is presented in Figure 8. Through detailed analysis, the integration of user attributes, such as age, gender, and movie genre preferences, effectively mitigates the loss of user-specific latent feature information derived from matrix factorization, resulting in a notable reduction in prediction error (RMSE). However, when the weight assigned to latent features obtained via the ALS algorithm becomes excessively high, the model experiences slight performance degradation, likely due to over-reliance on user-specific features. This finding underscores the critical role of user preference features in enhancing the overall effectiveness of recommendation systems.



(a) RMSE for 1M and 100K dataset



(b) Evaluation indicators and F1 score for 1M and 100K dataset

FIGURE 9. Weight value analysis in step 4

The similarity weight diagram illustrating the relationship between movie text latent feature (LL) similarity in Step 3 and the hidden movie feature vector in Step 1 is shown in Figure 8. The incorporation of movie text features with appropriately balanced weights

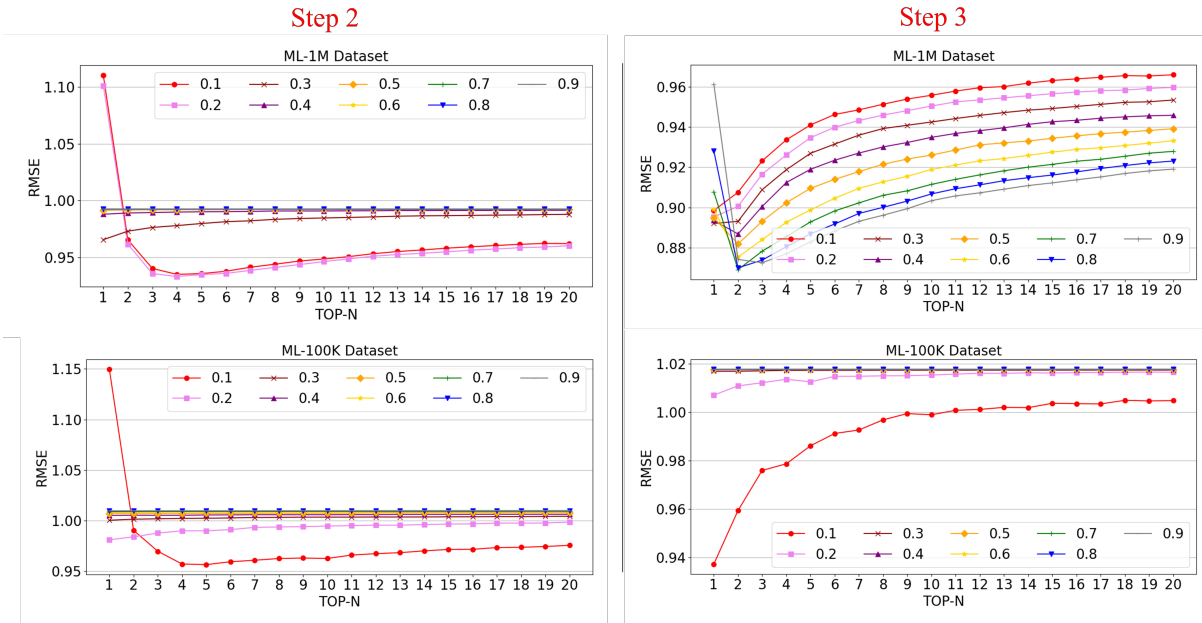


FIGURE 8. Weight value analysis in step 2 & step 3

helps to offset the loss of latent movie feature information, thereby improving the overall performance of the recommendation system. These findings emphasize the importance of maintaining a moderate weight for text features to avoid any negative impact on recommendation accuracy and highlight the significant role of text features in augmenting the functionality and precision of the system.

α within the range of 0 to 1 with an interval of 0.1. The model will compute four evaluation metrics using different weight values α in the ML-1M and ML-100K datasets via formula (25). The computed results are presented in Figure 9.

TABLE 11. Performance metrics value for the proposed method on target dataset.

Dataset	α	Precision	Recall	F1	RMSE
ML-1M	0.3	0.904	0.966	0.935	0.824
ML-100K	0.3	0.894	0.927	0.91	0.881

When the parameter α is set to 0.3, the model achieves optimal performance on both datasets, as detailed in Table 11. Comparative analysis of this model's performance against models reported in other studies is presented in Table 12.

V. CONCLUSIONS

We propose a hybrid movie recommender system combining content-based filtering (CBF) and collaborative filtering (CF) based on matrix factorization to address cold-start and sparsity challenges. Besides, we introduce a weighting scheme for the movie text features and movie

implicit features composed of LL values to compensate for the movie information loss during matrix decomposition; user demographic features such as age, gender, and category preferences are weighted alongside the user's implicit features to mitigate the loss of user information in the decomposition process.

Our experiments demonstrate that LL features offer advantages over TF-IDF features. Furthermore, the experiment evaluates different similarity metrics to determine the optimal measurement method for calculating similarities between implicit user/movie features. The results reveal that our proposed model exhibits lower RMSE error and improved precision, recall, and F1 value compared to existing recommendation systems.

However, the model still has some shortcomings. First, for new users, the system requires accurate information regarding their age, gender, and category preferences. If this information is incorrect, the recommendation system cannot provide precise recommendations. Second, the prediction results from steps 2 and 3 of the model are weighted using the mean, which overlooks the possibility that one prediction rating matrix might be more significant.

To address these limitations, the following strategy can be considered: leveraging the initial behavior of new users (such as browsing history or click behavior) to infer whether their age, gender, and movie type preferences align with the initial recommendation content. This allows for the adjustment of weights between content-based filtering (CBF) and collaborative filtering (CF) within the hybrid recommendation algorithm, ultimately providing users with more personalized rec-

TABLE 12. Comparison with other methods

Model	Method presentation	ML-100K				ML-1M			
		Prec.	Rec.	F1	RMSE	Prec.	Rec.	F1	RMSE
ICFTU [13]	An improved collaborative recommendation algorithm is proposed by combining the Term Frequency-Inverse Document Frequency (TF-IDF) method and user characteristics model	-	-	-	0.941	-	-	-	0.976
SVD++ [14]	Implicit information feedback is used by SVD++. Implicit feedback is a term that refers to any type of data that will be examined to provide predictable results and hence serve as a guide for users.	-	-	-	-	-	-	-	0.856
IPWR with SD [21]	An improved PCC was weighted with rating preference behavior. In the IPWR similarity measure method, user rating preference behavior is modeled as a Cosine function of user averages and variance or SD.	0.617	0.431	0.506	0.993	0.616	0.357	0.409	0.959
ALS-WR [14]	The weighted Regularization for Alternating Least Squares introduces weighted regularization based on the original ALS algorithm.	-	-	-	-	-	-	-	0.87
UICF [27]	the UICF model combines user and item collaborative filtering with linear regression to tackle rating matrix sparsity and scalability, demonstrating superior performance in real-world datasets.	-	-	-	-	0.85	0.819	0.855	-
DLCRS [28]	DLCRS model concatenates the user and movie identities as one-hot. A fully connected layer deep learning neural network is used as the embedding layer to learn the lower-dimension the embedding process is formulated.	-	-	-	0.917	-	-	-	0.903
CFTF [29]	A collaborative filtering model was proposed to incorporate temporal effects and address the dynamic nature of user-item interactions.	-	-	-	0.9119	-	-	-	0.853
SCOR-DTEC [30]	The proposed Dual Training Error based Correction (DTEC) method enhances recommender systems by correcting prediction errors from both user and item perspectives, improving accuracy on real-world datasets.	-	-	-	-	-	-	-	0.859
Proposed model	A novel hybrid model that integrates matrix factorization with additional features. Besides, it leverages weighted similarity measures and employs advanced log-likelihood text mining techniques.	0.897	0.947	0.92	0.883	0.912	0.962	0.937	0.828

ommendations. This approach can mitigate the impact of incorrect information regarding the age, gender, and movie category preferences of new users while enhancing the personalization of recommendations through contextual adjustments. Furthermore, it helps prevent the loss of important information that may occur due to static averaging.

REFERENCES

- [1] N. Kannikaklang, S. Wongthanavasu, and W. Thamviset, "A hybrid recommender system for improving rating prediction of movie recommendation," in 2022 19th International Joint Conference on Computer Science and Software Engineering (JCSSE). IEEE, 2022, pp. 1–6.
- [2] P. G. Padti, K. Hegde, and P. Kumar, "Hybrid movie recommender system," International Journal of Research in Engineering, Science and Management, vol. 4, no. 7, pp. 311–314, 2021.
- [3] N. Musthafa, C. Raji et al., "Hybrid recommender system using k-means clustering," in 2022 8th International Conference on Advanced Computing and Communication Systems (ICACCS), vol. 1. IEEE, 2022, pp. 625–630.
- [4] M. S. Medikonduru, P. Bypureddy, V. P. Kancharla, V. N. H. Y. P. Kamarajugadda, and L. C. Attluri, "Structured analysis on movie recommendation system using machine learning," in 2022 6th International Conference on Intelligent Computing and Control Systems (ICICCS). IEEE, 2022, pp. 1297–1303.
- [5] Z. Strömquist, "Matrix factorization in recommender systems: How sensitive are matrix factorization models to sparsity?" 2018.
- [6] S. Bag, S. K. Kumar, and M. K. Tiwari, "An efficient rec-

- ommendation generation using relevant jaccard similarity," 731
Information Sciences, vol. 483, pp. 53–64, 2019. 732
- [7] A.-Q. Tian, X.-Y. Wang, H. Xu, J.-S. Pan, V. Snášel, and 733
H.-X. Lv, "Multi-objective optimization model for railway 734
heavy-haul traffic: addressing carbon emissions reduction 735
and transport efficiency improvement," Energy, vol. 294, p. 736
130927, 2024. 737
- [8] J.-S. Pan, A.-Q. Tian, V. Snášel, L. Kong, and S.-C. Chu, 738
"Maximum power point tracking and parameter estimation 739
for multiple-photovoltaic arrays based on enhanced pigeon- 740
inspired optimization with taguchi method," Energy, vol. 251, 741
p. 123863, 2022. 742
- [9] A.-Q. Tian, F.-F. Liu, and H.-X. Lv, "Snow geese algo- 743
rithm: A novel migration-inspired meta-heuristic algorithm 744
for constrained engineering optimization problems," Applied 745
Mathematical Modelling, vol. 126, pp. 327–347, 2024. 746
- [10] M. Mngomezulu and R. Ajoodha, "A content-based collabo- 747
rative filtering movie recommendation system using keywords 748
extractions," in 2022 International Conference on Engineering 749
and Emerging Technologies (ICEET). IEEE, 2022, pp. 1–6. 750
- [11] Á. González, F. Ortega, D. Pérez-López, and S. Alonso, "Bias 751
and unfairness of collaborative filtering based recommender 752
systems in movielens dataset," IEEE Access, vol. 10, pp. 753
68 429–68 439, 2022. 754
- [12] M. Gupta, A. Thakkar, V. Gupta, D. P. S. Rathore et al., 755
"Movie recommender system using collaborative filtering," in 756
2020 international conference on electronics and sustainable 757
communication systems (ICESC). IEEE, 2020, pp. 415–420. 758
- [13] J. Ni, Y. Cai, G. Tang, and Y. Xie, "Collaborative filtering 759
recommendation algorithm based on tf-idf and user charac- 760
teristics," Applied Sciences, vol. 11, no. 20, p. 9554, 2021. 761
- [14] A. Priyati, A. D. Laksito, and H. Sismoro, "The comparison 762
study of matrix factorization on collaborative filtering rec- 763
ommender system," in 2022 5th International Conference on 764
Information and Communications Technology (ICOACT). IEEE, 2022, pp. 177–182. 765
- [15] Z. Wang, J. Huang, and H. Sun, "Improved als recommenda- 766
tion algorithm based on interest-value model," in Journal of 767
Physics: Conference Series, vol. 2219, no. 1. IOP Publishing, 768
2022, p. 012047. 769
- [16] Y. Li, X. Chu, D. Tian, J. Feng, and W. Mu, "Customer seg- 770
mentation using k-means clustering and the adaptive particle 771
swarm optimization algorithm," Applied Soft Computing, 772
vol. 113, p. 107924, 2021. 773
- [17] W. Kamlor and K. Cosh, "Product discovery via recommen- 774
dation based on user comments," in 2014 6th International 775
Conference on Knowledge and Smart Technology (KST). 776
IEEE, 2014, pp. 41–45. 777
- [18] P. Rayson and R. Garside, "Comparing corpora using fre- 778
quency profiling," in The workshop on comparing corpora, 779
2000, pp. 1–6. 780
- [19] D. Meyer, "How exactly does word2vec work," Uoregon. Edu, 781
Brocade. Com, pp. 1–18, 2016. 782
- [20] R. Turnip, D. Nurjanah, and D. S. Kusumo, "Hybrid rec- 783
ommender system for learning material using content-based 784
filtering and collaborative filtering with good learners' rat- 785
ing," in 2017 IEEE Conference on e-Learning, e-Management, 786
and e-Services (IC3e). IEEE, 2017, pp. 61–66. 787
- [21] M. Ayub, M. A. Ghazanfar, Z. Mehmood, T. Saba, R. Al- 788
harbey, A. M. Munshi, and M. A. Alrige, "Modeling user 789
rating preference behavior to improve the performance of 790
the collaborative filtering based recommender systems," PloS 791
one, vol. 14, no. 8, p. e0220129, 2019. 792
- [22] D. Li and Y. Tian, "Survey and experimental study on metric 793
learning methods," Neural networks, vol. 105, pp. 447–462, 794
2018. 795
- [23] S. Okyay and S. Aygun, "Experimental interpretation of 796
adequate weight-metric combination for dynamic user-based 797
collaborative filtering," PeerJ Computer Science, vol. 7, p. 798
e784, 2021. 799
- [24] H. Kumaar, S. Srikumaran, S. Veni et al., "Content- 800
based movie recommender system using keywords and plot 801
overview," in 2022 International Conference on Wireless 802
Communications Signal Processing and Networking (WiSP- 803
NET). IEEE, 2022, pp. 49–53. 804
- [25] J. Zhang, Y. Wang, Z. Yuan, and Q. Jin, "Personalized real- 805
time movie recommendation system: Practical prototype and 806
evaluation," Tsinghua Science and Technology, vol. 25, no. 2, 807
pp. 180–191, 2019. 808
- [26] K. Ong, S.-C. Haw, and K.-W. Ng, "Deep learning based- 809
recommendation system: an overview on models, datasets, 810
evaluation metrics, and future trends," in Proceedings of the 811
2019 2nd international conference on computational intelli- 812
gence and intelligent systems, 2019, pp. 6–11. 813
- [27] M. F. Aljunid and M. D. Huchaiah, "An efficient hybrid rec- 814
ommendation model based on collaborative filtering recom- 815
mender systems," CAAI Transactions on Intelligence Tech- 816
nology, vol. 6, no. 4, pp. 480–492, 2021. 817
- [28] M. F. Aljunid and M. Dh, "An efficient deep learning 818
approach for collaborative filtering recommender system," 819
Procedia Computer Science, vol. 171, pp. 829–836, 2020. 820
- [29] G. Behera and N. Nain, "Collaborative filtering with tem- 821
poral features for movie recommendation system," Procedia 822
Computer Science, vol. 218, pp. 1366–1373, 2023. 823
- [30] C. Panagiotakis, H. Papadakis, A. Papagrigoriou, and 824
P. Fragopoulou, "Improving recommender systems via a dual 825
training error based correction approach," Expert Systems 826
with Applications, vol. 183, p. 115386, 2021. 827



YANG YONGMAO Born in 1998, he obtained a bachelor's degree in computer electronics and technology in Nanjing, China in 2020, and is currently studying as a graduate student in computer engineering at Chiang Mai University, Thailand, Research interests are machine learning, recommender systems and natural language processing.



KAMPOL WORADIT (Member, IEEE) received the B.Eng. (Hons.) and Ph.D. degrees in electrical engineering from Chulalongkorn University, Thailand, in 2002 and 2010, respectively. He was a Visiting Student with the Laboratory for Information and Decision Systems, Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, from 2007 to 2008, and was on a Postgraduate Research Attachment with the Modulation and Coding Department, Institute for Infocomm Research, Singapore. He was a Faculty Member with the Department of Electrical Engineering, Srinakharinwirot University, Thailand, from 2010 to 2020. He is currently a Faculty Member with the Department of Computer Engineering, at Chiang Mai University, Thailand. His research interests include wireless communications, the Internet of Things, and physical layer security. He received the ECTI-CON Best Paper Award in 2019 and the Royal Golden Jubilee Scholarship from the Thailand Research Fund for his Ph.D. Program.



ments Engineering.

ASSOC. PROF. KENNETH COSH graduated in Business Management from Lancaster University (UK), before returning there to gain his doctorate in Computing. Since then he has been an academic in Thailand for the past 20 years mostly working in the Computer Engineering department at Chiang Mai University. His research interests include Natural Language Processing, Algorithms, Software Engineering, and Require-

...